

UT 02.4 – Estados y eventos en React

1. Dónde estamos y qué vamos a ver

Este tema consolida y amplía lo que ya habéis usado en UT 02.2 y 02.3 (`useState` y eventos), pero ahora el objetivo es **entender por qué funcionan así**.

1.1 Objetivos

Al terminar el tema serás capaz de:

- Explicar por qué una variable normal no sirve para cambiar la pantalla y qué aporta el estado.
- Describir el ciclo *evento* → *actualización del estado* → *nuevo renderizado* → *DOM actualizado*.
- Entender que el estado es una “instantánea” por renderizado y por qué el valor no cambia justo después de llamar a `setX`.
- Actualizar el estado sin mutarlo, también cuando depende del valor anterior.
- Decidir qué datos deben ser estado y cuáles se pueden calcular.
- Gestionar eventos con comodidad: propagación, `preventDefault` y teclado.
- Construir un formulario sencillo con campos controlados.
- Compartir estado entre componentes elevándolo al padre.

1.2 Cómo leer los ejemplos

Todos los ejemplos están en TSX y siguen la regla de UT 02.3: se deja que TypeScript **infera** los tipos siempre que pueda y solo se anotan cuando hace falta (estado que empieza en `null` o `[]`, manejadores extraídos a una función, props). Si algún tipo te resulta poco familiar, repasa los apuntes de UT 02.3.

2. Repaso exprés: lo que ya sabes

Antes de profundizar, comprueba que dominas estas piezas. Si alguna fila te resulta nueva, repásala en UT 02.2 o 02.3.

Concepto	Ejemplo	Visto en
Manejador de evento: se pasa la función, no su resultado	<code><button onClick={cerrar}></code>	UT 02.2
Manejador en línea	<code><button onClick={() => setVisible(false)}></code>	UT 02.2
Estado con <code>useState</code>	<code>const [visible, setVisible] = useState(true)</code>	UT 02.2
Eventos propios mediante una <i>prop</i> de función	<code><Alerta onClose={() => console.log("cerrada")} /></code>	UT 02.2
Tipo inferido del estado	<code>useState(0)</code> es <code>number</code>	UT 02.3

Concepto	Ejemplo	Visto en
Tipo explícito cuando el valor inicial no basta	<code>useState<Usuario null>(null)</code>	UT 02.3
Evento tipado en un manejador aparte	<code>(e: ChangeEvent<HTMLInputElement>)</code>	UT 02.3

Un componente que reúne casi todo lo anterior es esta alerta que se puede cerrar:

```
import { useState } from "react";

interface PropsAlerta {
  titulo: string;
  alCerrar?: () => void; // evento propio, opcional
}

export function Alerta({ titulo, alCerrar }: PropsAlerta) {
  const [visible, setVisible] = useState(true); // boolean inferido

  if (!visible) {
    return null;
  }

  function manejarCierre() {
    setVisible(false);
    alCerrar?.(); // se llama solo si existe
  }

  return (
    <div role="alert">
      <span>{titulo}</span>
      <button onClick={manejarCierre}>Cerrar</button>
    </div>
  );
}
```

Con esto ya puedes cambiar la pantalla al hacer clic. En las secciones siguientes veremos **por qué** funciona, **qué** conviene guardar en el estado y **cómo** evitar los errores más habituales.

3. Por qué hace falta el estado

Con componentes y *props* solo se pueden construir interfaces **estáticas**. Para entender qué aporta el estado, vamos a intentar resolver un problema real en cuatro intentos, de peor a mejor. El problema: **un campo de correo que muestra un mensaje de error cuando el usuario sale del campo (evento `blur`) y la dirección no es válida**.

Intento 1: componente estático

```
function EntradaCorreo() {
  return (
    <div>
      <input placeholder="Tu correo electrónico" type="email" />
      <p>La dirección de correo introducida no es válida.</p>
    </div>
  );
}
```

El mensaje se muestra **siempre**. No hay forma de que cambie en función de lo que hace el usuario.

Intento 2: tocar el DOM a mano (código imperativo)

La solución que se nos ocurre viniendo de JavaScript clásico es buscar los elementos y modificarlos:

```
function EntradaCorreo() {
  return (
    <div>
      <input placeholder="Tu correo electrónico" type="email" />
      <p></p>
    </div>
  );
}
```

// ❌ No lo hagas en React

```
const entrada = document.querySelector("input");
const parrafoError = document.querySelector("p");
```

```
function evaluarCorreo(evento: Event) {
  const correo = (evento.target as HTMLInputElement).value;
  if (correo.trim() === "" || !correo.includes("@")) {
    parrafoError!.textContent = "La dirección no es válida.";
  } else {
    parrafoError!.textContent = "";
  }
}
```

```
entrada?.addEventListener("blur", evaluarCorreo);
```

No funciona porque ese código se ejecuta **antes** de que React pinte los elementos en el DOM: `querySelector` devuelve `null`. Aunque lo forzáramos con un `setTimeout`, seguiría siendo un mal enfoque: es **código imperativo** (escribes los pasos para modificar el DOM) y React es **declarativo** (describes cómo debe verse la interfaz y React decide cómo llegar a ella).

	Imperativo	Declarativo (React)
Qué escribes	Los pasos para cambiar el DOM	El resultado deseado según los datos
Quién modifica el DOM	Tú	React
Ejemplo	<code>parrafo.textContent = "Error"</code>	<code>{hayError && <p>Error</p>}</code>
Al crecer la aplicación	Cada vez más difícil de mantener	Escala bien

Intento 3: variable normal y evento de React

El siguiente paso es dejar que el propio componente se encargue de sus eventos y de sus datos, escuchando el evento directamente en el JSX con `onBlur`:

```
function EntradaCorreo() {
  let mensajeError = ""; // variable normal
```

```
function evaluarCorreo(evento: React.FocusEvent<HTMLInputElement>) {
  const correo = evento.target.value;
  mensajeError =
    correo.trim() === "" || !correo.includes("@")
      ? "La dirección de correo introducida no es válida."
      : "";
  console.log("mensajeError vale ahora:", mensajeError); // aquí sí cambia...
}

return (
  <div>
    <input type="email" placeholder="Tu correo electrónico" onBlur={evaluarCorreo} />
    <p>{mensajeError}</p>
  </div>
);
}
```

Ahora el evento está bien conectado y la variable cambia (lo comprobamos con `console.log`), pero **la pantalla no se actualiza**. Hay dos motivos:

1. React **no vigila** las variables normales, así que no se entera de que `mensajeError` ha cambiado.
2. Aunque el componente se volviera a ejecutar, `let mensajeError = ""` empezaría de cero: las variables locales **no sobreviven** entre ejecuciones.

Necesitamos un dato que React **recuerde** entre ejecuciones y que, al cambiar, **avise** a React de que hay que actualizar la pantalla. Eso es el estado.

Intento 4: estado con `useState`

```
import { useState } from "react";

function EntradaCorreo() {
  const [mensajeError, setMensajeError] = useState("");

  function evaluarCorreo(evento: React.FocusEvent<HTMLInputElement>) {
    const correo = evento.target.value;
    if (correo.trim() === "" || !correo.includes("@")) {
      setMensajeError("La dirección de correo introducida no es válida.");
    } else {
      setMensajeError("");
    }
  }

  return (
    <div>
      <input type="email" placeholder="Tu correo electrónico" onBlur={evaluarCorreo} />
      {mensajeError && <p>{mensajeError}</p>}
    </div>
  );
}
```

El componente apenas ha cambiado respecto al intento 3, pero ahora:

- `mensajeError` es un dato que **React guarda** entre renderizados.
- `setMensajeError` **actualiza ese dato y avisa a React** de que debe volver a ejecutar el componente.
- El JSX **declara** qué se ve según el estado: sin mensaje, no hay `<p>`.

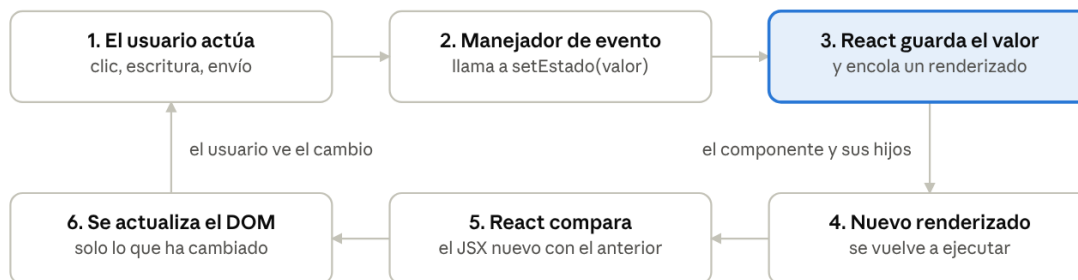
Idea clave. El estado es un dato que, cuando cambia, obliga a React a volver a ejecutar el componente y a actualizar la pantalla si hace falta. Una variable normal no tiene ese poder.

4. Cómo se renderiza un componente

Para manejar bien el estado hay que tener claro un modelo mental: **renderizar un componente es ejecutar su función**. React la llama la primera vez para saber qué dibujar y la vuelve a llamar cada vez que cambia el estado.

4.1 El ciclo del estado

Cambiar el estado es lo que provoca un nuevo renderizado



Si el valor nuevo es igual al anterior (Object.is), React no vuelve a renderizar.

ciclo del estado · 6 pasos

El paso 3 es el decisivo: solo cuando React recibe un valor nuevo a través de la función de actualización se inicia el ciclo. Modificar una variable normal no avisa a nadie. Esto enlaza con lo que viste en UT 02.1 sobre el DOM virtual: React compara el resultado nuevo con el anterior y solo toca el DOM real donde hay diferencias.

Además del propio componente, React vuelve a ejecutar **sus hijos** (que pueden recibir el estado por *props*). Puedes comprobarlo con la pestaña *Profiler* de las React DevTools que usaste en UT 02.2.

4.2 El estado es una “instantánea”

Dentro de una misma ejecución del componente, el valor del estado **no cambia**: es como una fotografía. Llamar a `setX` no modifica la variable que ya tienes; pide a React un renderizado *nuevo* en el que `useState` devolverá el valor actualizado.

```
function Contador() {
  const [cuenta, setCuenta] = useState(0);

  function manejarClic() {
    console.log("antes:", cuenta); // 0 en el primer clic
    setCuenta(cuenta + 1);
    console.log("después:", cuenta); // ⚠ sigue siendo 0, no 1
  }

  return <button onClick={manejarClic}>Pulsado {cuenta} veces</button>;
}
```

En el primer clic verás antes: 0 y después: 0. El 1 solo existe en el **siguiente** renderizado, cuando el botón muestre Pulsado 1 veces.

Por eso cuenta se puede declarar con const: en cada ejecución del componente se crea una constante **nueva** con el valor que React guardó, y la anterior se descarta.

Ejecución del componente	Valor de cuenta	Qué ve el usuario
1. ^a (primer renderizado)	0	Pulsado 0 veces
2. ^a (tras el primer clic)	1	Pulsado 1 veces
3. ^a (tras el segundo clic)	2	Pulsado 2 veces

***Para pensar.** Si dentro de manejarClic pones `setTimeout(() => console.log(cuenta), 3000)` y pulsas dos veces seguidas, ¿qué valores se imprimen? El manejador de cada clic pertenece a una ejecución distinta del componente y recuerda su foto: se imprimen 0 y 1, no 2.*

4.3 Componentes puros y StrictMode

En UT 02.1 viste que `main.tsx` envuelve la aplicación en `<StrictMode>`. En desarrollo, ese componente hace que React **ejecute dos veces** cada componente para detectar errores. Si ves un `console.log` duplicado, no es un fallo: es `StrictMode` comprobando que tu componente es **puro**, es decir, que con las mismas *props* y el mismo estado devuelve siempre el mismo JSX y no modifica nada fuera de sí mismo.

```
let visitas = 0; // ❌ variable fuera del componente

function Contador() {
  visitas = visitas + 1; // ❌ modificar algo externo durante el renderizado
  return <p>Visitas: {visitas}</p>; // en desarrollo salta de 2 en 2
}
```

La solución es no tocar nada externo mientras se renderiza: los cambios van en los **manejadores de eventos** y en el **estado**. En producción `StrictMode` no hace nada, así que no afecta al rendimiento de la aplicación final.

5. Actualizar el estado correctamente

La función de actualización (`setX`) no “modifica” el estado: **entrega a React un valor nuevo** que sustituirá al anterior en el próximo renderizado. De ahí salen tres reglas que evitan la mayoría de los errores.

5.1 Regla 1: el valor nuevo reemplaza al anterior

`useState` **no fusiona** objetos: lo que pasas a `setX` sustituye por completo al estado. Por eso, con objetos, hay que conservar las propiedades que no cambian.

```
interface DatosAcceso {
  correo: string;
  contraseña: string;
}

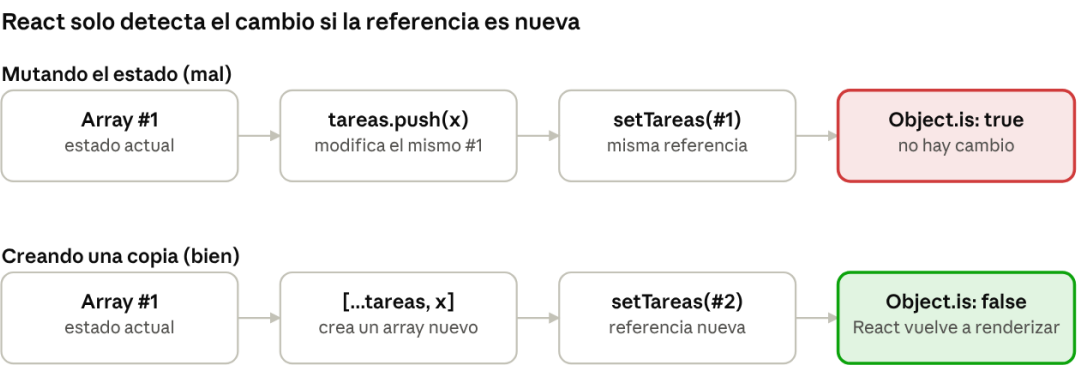
const [datos, setDatos] = useState<DatosAcceso>({ correo: "", contraseña: "" });

setDatos({ correo: "ana@correo.es" });
// ~ Error de TypeScript: falta 'contraseña'
```

`setDatos({ ...datos, correo: "ana@correo.es" }); // ✅ copia las demás propiedades`
En JavaScript, la primera línea se ejecutaría sin avisar y se **perdería** la contraseña. Es un buen ejemplo de error que TypeScript detecta por ti. El operador `...` (*spread*) copia todas las propiedades del objeto y la propiedad que escribes después sobrescribe solo la que cambia.

5.2 Regla 2: nunca modifiques el estado, crea una copia

React decide si debe volver a renderizar **comparando referencias**: mira si el valor que le pasas es *el mismo objeto* que tenía (con `Object.is`). Si modificas un array u objeto existente y se lo devuelves, la referencia es la misma y React cree que no ha cambiado nada.



mutar o copiar el estado · 2 recorridos

```
const [tareas, setTareas] = useState<string[]>(["Estudiar", "Practicar"]);

function añadirMal() {
  tareas.push("Repasar"); // ❌ modifica el array existente
  setTareas(tareas);      // misma referencia: la pantalla NO se actualiza
}

function añadirBien() {
  setTareas([...tareas, "Repasar"]); // ✅ array nuevo, referencia nueva
}
```

Estas son las equivalencias que conviene memorizar (los ejemplos completos con listas de tareas están en los apuntes de UT 02.3):

Operación	❌ Modifica (evitar)	✅ Crea una copia
Añadir	<code>push, unshift</code>	<code>[...lista, nuevo]</code>
Eliminar	<code>splice, pop</code>	<code>lista.filter((x) => x.id !== id)</code>
Cambiar un elemento	<code>lista[0] = x</code>	<code>lista.map((x) => x.id === id ? { ...x, hecha: true } : x)</code>
Ordenar	<code>sort, reverse</code>	<code>[...lista].sort()</code>
Objeto dentro de un objeto	<code>usuario.direccion.ciudad =</code>	<code>{ ...usuario, direccion: {</code>

Operación	❌ Modifica (evitar)	✅ Crea una copia
	"Gijón"	...usuario.direccion, ciudad: "Gijón" } }

5.3 Regla 3: si depende del valor anterior, usa la forma funcional

React **agrupa** las actualizaciones de estado de un mismo evento y las procesa juntas, de modo que todas lean la misma "instantánea" (sección 4.2):

```
function sumarTres() {
  // ❌ Las tres llamadas leen cuenta = 0 → el resultado final es 1
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
}

function sumarTresBien() {
  // ✅ Cada función recibe el resultado de la anterior → el resultado final es 3
  setCuenta((anterior) => anterior + 1);
  setCuenta((anterior) => anterior + 1);
  setCuenta((anterior) => anterior + 1);
}
```

Cuando pasas una función a setX, React **no la guarda como estado**: la llama por ti, le entrega el valor más reciente y guarda lo que devuelve. TypeScript infiere el tipo de anterior.

- Si el valor nuevo **depende del anterior** (contadores, añadir a una lista, alternar un booleano), usa `setX((anterior) => ...)`.
- Si procede de **otra fuente** (el texto de un input, un valor fijo), pásalo directamente: `setNombre(e.target.value)`.

En objetos, la forma funcional queda así:

```
setDatos((anteriores) => ({ ...anteriores, correo: nuevoCorreo }));
```

5.4 Un solo estado con varios campos o varios estados

Cuando un componente necesita guardar varios datos hay dos opciones y ambas son válidas:

Opción	Cuándo conviene	Ejemplo
Varios useState	Los datos son independientes entre sí	const [correo, setCorreo] = useState("") y const [aceptado, setAceptado] = useState(false)
Un useState con un objeto	Los datos cambian juntos o forman una unidad	useState<DatosAcceso>({ correo: "", contraseña: "" })

Si un componente acumula muchos estados, suele ser señal de que conviene dividirlo en componentes más pequeños. En el capítulo 3 se verá `useReducer`, otra herramienta para estados con mucha lógica.

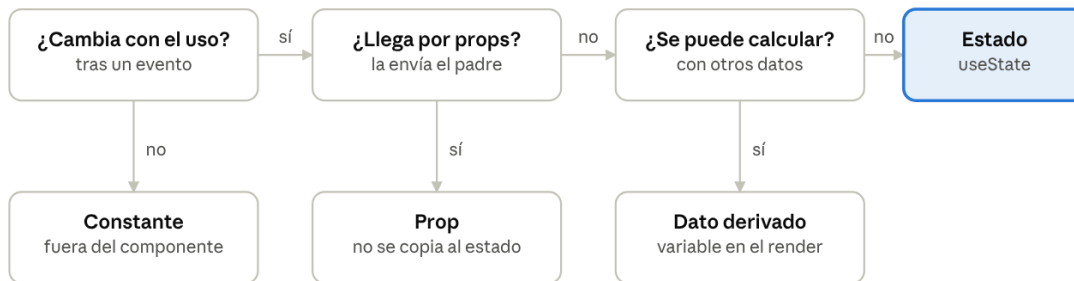
6. Qué merece ser estado

Cada dato que guardas en el estado es un dato más que mantener sincronizado. La regla de oro es guardar **lo mínimo imprescindible** y calcular todo lo demás.

6.1 Tres preguntas

Antes de crear un `useState`, pregúntate esto sobre el dato:

Solo es estado lo que cambia, no llega por props y no se puede calcular



¿es estado? · 3 preguntas

Leyendo el esquema de izquierda a derecha: si el dato no cambia, es una constante; si te lo envía el padre, es una *prop* (no la copies al estado); si puedes obtenerlo a partir de otros datos, es un **dato derivado**; solo si no se cumple ninguna de las tres, es estado.

6.2 Datos derivados

Un dato derivado es una variable normal calculada en el cuerpo del componente. Como React vuelve a ejecutar el componente tras cada cambio de estado (sección 4), el dato siempre está al día sin que tengas que sincronizarlo.

```
function ContadorCaracteres() {
  const [texto, setTexto] = useState("");

  // ✅ Derivados: se recalculan en cada renderizado
  const numeroCaracteres = texto.length;
  const superaLimite = numeroCaracteres > 140;

  return (
    <>
      <textarea value={texto} onChange={(e) => setTexto(e.target.value)} />
      <p style={{ color: superaLimite ? "crimson" : "inherit" }}>
        Caracteres: {numeroCaracteres} / 140
      </p>
    </>
  );
}
```

`numeroCaracteres` puede ser una `const` aunque cambie: en cada renderizado se crea una nueva a partir del `texto` actual.

Otro caso muy habitual es filtrar una lista. Guarda **el criterio** en el estado y **calcula** la lista filtrada:

```

interface Producto {
  id: number;
  nombre: string;
}

function ListaProductos({ productos }: { productos: Producto[] }) {
  const [busqueda, setBusqueda] = useState("");

  // ✅ Derivado: no hace falta otro estado con la lista filtrada
  const visibles = productos.filter((p) =>
    p.nombre.toLowerCase().includes(busqueda.toLowerCase())
  );

  return (
    <>
      <input value={busqueda} onChange={(e) => setBusqueda(e.target.value)} />
      <ul>
        {visibles.map((p) => (
          <li key={p.id}>{p.nombre}</li>
        ))}
      </ul>
    </>
  );
}

```

6.3 Dos errores típicos

Duplicar información. Guardar visibles en su propio useState y actualizarlo a mano obliga a mantener dos datos sincronizados: si olvidas uno, la pantalla muestra algo inconsistente.

Copiar una prop al estado. El valor que pasas a useState solo se usa en el **primer** renderizado:

```

function Saludo({ nombre }: { nombre: string }) {
  const [copia, setCopia] = useState(nombre); // ❌ solo lee 'nombre' la primera vez
  return <p>Hola, {copia}</p>; // si el padre cambia 'nombre', no se entera
}

function SaludoBien({ nombre }: { nombre: string }) {
  return <p>Hola, {nombre}</p>; // ✅ usa la prop directamente
}

```

En el capítulo 10 se clasifican los tipos de estado (local, derivado, compartido, de servidor...) y se verá useMemo para evitar recálculos costosos de datos derivados.

7. Eventos en profundidad

Ya sabes conectar un manejador con onClick. Ahora conviene conocer qué recibe ese manejador, qué eventos existen y cómo se comportan.

7.1 El objeto evento

Cuando ocurre un evento, React llama al manejador y le pasa un **objeto evento** (un envoltorio multinaavegador del evento nativo, llamado evento sintético). Estas son las propiedades más útiles:

Propiedad	Qué contiene	Ejemplo
target	El elemento donde se originó el evento	e.target.value en un input
currentTarget	El elemento al que está unido el manejador	e.currentTarget.textContent
type	El nombre del evento	"click", "change"
key	La tecla pulsada (eventos de teclado)	"Enter", "Escape"
clientX, clientY	Posición del ratón (eventos de ratón)	e.clientX
preventDefault()	Cancela el comportamiento por defecto del navegador	Evita recargar al enviar un formulario
stopPropagation()	Impide que el evento llegue a los elementos padre	Ver sección 7.3

El tipado de cada evento (ChangeEvent, MouseEvent, KeyboardEvent...) se vio en UT 02.3. Recuerda el truco: si escribes el manejador en línea, TypeScript lo infiere; si lo extraes a una función, tienes que anotarlo.

7.2 Eventos habituales

Evento	Cuándo se dispara	Uso típico
onClick	Al hacer clic	Botones, tarjetas, enlaces
onChange	En cada cambio del valor (en React, en cada pulsación)	Campos de texto, casillas, select
onBlur	Al perder el foco	Validar al salir del campo
onFocus	Al ganar el foco	Mostrar ayudas
onKeyDown	Al pulsar una tecla	Atajos, enviar con Enter
onSubmit	Al enviar un formulario	Gestionar el formulario completo
onMouseEnter / onMouseLeave	El ratón entra o sale	Resaltados, menús

Una diferencia importante con JavaScript puro: en React, onChange se dispara **en cada pulsación**, no al terminar de escribir (en el DOM nativo ese comportamiento es el del evento `input`). Si solo quieres reaccionar al terminar, usa `onBlur`.

7.3 Propagación de eventos (*bubbling*)

Cuando haces clic en un elemento, el evento **no se queda ahí**: después de ejecutar el manejador del propio elemento, **sube** por los elementos que lo contienen y ejecuta sus manejadores. A este comportamiento se le llama *burbujeo*.

Un clic en el botón también llega al div que lo contiene



Orden de ejecución

1. borrar (el botón)
2. abrirDetalle (el div)

Con `e.stopPropagation()` dentro de borrar, el paso 2 ya no se ejecuta.

propagación de un evento · botón dentro de un div

```
function Tarjeta({ id }: { id: number }) {
  function abrirDetalle() {
    console.log("Abriendo el detalle de", id);
  }

  function borrar(e: React.MouseEvent<HTMLButtonElement>) {
    e.stopPropagation(); // sin esta línea, el clic también abriría el detalle
    console.log("Borrando", id);
  }

  return (
    <div onClick={abrirDetalle}>
      <p>Tarjeta {id}</p>
      <button onClick={borrar}>Borrar</button>
    </div>
  );
}
```

Si quitas `e.stopPropagation()` verás los dos mensajes en la consola: primero el del botón y después el del div.

7.4 Cancelar el comportamiento por defecto

Algunos elementos HTML hacen algo por sí solos: un `<form>` recarga la página al enviarse y un `<a>` navega a su destino. Con `e.preventDefault()` se cancela ese comportamiento para gestionarlo desde React (el ejemplo completo está en la sección 8).

```
function manejarEnlace(e: React.MouseEvent<HTMLAnchorElement>) {
  e.preventDefault(); // el navegador ya no navega
  console.log("Enlace pulsado, pero no navegamos");
}
```

No confundas `preventDefault()` (cancela la acción del navegador) con `stopPropagation()` (corta la subida del evento a los padres).

7.5 Pasar datos al manejador

Los manejadores solo reciben el evento. Para pasarles un dato propio (por ejemplo, el `id` del elemento sobre el que se hace clic) se usa una función flecha:

```
function ListaCompra({ articulos }: { articulos: { id: number; nombre: string }[] }) {
  function quitar(id: number) {
    console.log("Quitando el artículo", id);
  }

  return (
    <ul>
      {articulos.map((a) => (
        <li key={a.id}>
          {a.nombre}
          <button onClick={() => quitar(a.id)}>Quitar</button> {/*  */}
          {/* <button onClick={quitar(a.id)}>  se ejecuta al renderizar */}
        </li>
      ))}
    </ul>
  );
}
```

7.6 Eventos de teclado

Para detectar teclas se usa `onKeyDown` y se lee la propiedad `key`:

```
function BuscadorRapido() {
  const [texto, setTexto] = useState("");

  return (
    <input
      value={texto}
      onChange={(e) => setTexto(e.target.value)}
      onKeyDown={(e) => {
        if (e.key === "Enter") console.log("Buscando", texto);
        if (e.key === "Escape") setTexto(""); // limpiar el campo
      }}
    />
  );
}
```

7.7 Nombres: un convenio sencillo

- Los manejadores que **defines** en el componente se llaman `manejarXxx` (o `gestionarXxx`): `manejarClic`, `manejarCambioNombre`.
- Las *props* que **reciben** una función para un evento propio empiezan por `al` u `on`: `alCerrar`, `onCerrar`. Así se distinguen a simple vista de los atributos normales.

8. Formularios básicos con campos controlados

Los formularios son el mejor ejemplo para practicar estado y eventos juntos. En este tema nos quedamos con la versión más sencilla (un estado por campo y `onSubmit`); las herramientas avanzadas se estudian en el capítulo 9.

8.1 Campo controlado

Un campo es **controlado** cuando React gobierna su valor: el input **muestra** el estado (`value`) y **actualiza** el estado cuando el usuario escribe (`onChange`). Ese camino de ida y vuelta se conoce como *enlace bidireccional*.

```
function CampoBoletin() {
  const [correo, setCorreo] = useState("");

  return (
    <>
      <input
        type="email"
        placeholder="Tu correo electrónico"
        value={correo} // el estado se muestra
        onChange={(e) => setCorreo(e.target.value)} // lo que se escribe actualiza el
estado
      />
      <button onClick={() => setCorreo("")}>Restablecer</button>
    </>
  );
}
```

La ventaja se ve en el botón: al poner el estado a "", el campo se vacía también en pantalla. Si no pusiéramos `value={correo}`, el estado cambiaría pero el usuario seguiría viendo su texto.

Si pones `value` sin `onChange`, el campo queda de solo lectura y React avisa en la consola. Si solo quieres un valor inicial que el usuario pueda cambiar, usa `defaultValue`.

8.2 Cada tipo de campo, su propiedad

Campo	Propiedad que enlaza el estado	Qué se lee en el evento	Tipo del estado
Texto, correo, contraseña, textarea	<code>value</code>	<code>e.target.value</code>	string
Casilla (checkbox)	<code>checked</code>	<code>e.target.checked</code>	boolean
Lista desplegable (select)	<code>value</code>	<code>e.target.value</code>	string

8.3 Enviar el formulario

Para reaccionar al envío del formulario completo se usa `onSubmit` en el `<form>`. Por defecto el navegador recargaría la página, así que lo primero que se hace es llamar a `e.preventDefault()` (sección 7.4).

```
function AltaBoletin() {
  const [correo, setCorreo] = useState("");
  const [acepta, setAcepta] = useState(false);
  const [enviado, setEnviado] = useState(false);

  function manejarEnvio(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault(); // evita la recarga de la página
    console.log("Datos enviados:", { correo, acepta });
    setEnviado(true);
  }

  if (enviado) {
    return <p>¡Gracias por suscribirte, {correo}!</p>;
  }
}
```

```

return (
  <form onSubmit={manejarEnvio}>
    <div>
      <label htmlFor="correo">Tu correo electrónico</label>
      <input
        id="correo"
        type="email"
        value={correo}
        onChange={(e) => setCorreo(e.target.value)}
      />
    </div>
    <div>
      <input
        id="acepta"
        type="checkbox"
        checked={acepta}
        onChange={(e) => setAcepta(e.target.checked)}
      />
      <label htmlFor="acepta">Acepto los términos y condiciones</label>
    </div>
    <button type="submit" disabled={!acepta || correo === ""}>
      Suscribirse
    </button>
  </form>
);
}

```

Fíjate en tres detalles:

- `htmlFor` es la versión React del atributo `for` de `<label>` (en JavaScript `for` es una palabra reservada). Une la etiqueta con su campo gracias al `id`, y mejora la accesibilidad.
- El botón con `type="submit"` (o pulsar Enter dentro de un campo) es lo que dispara `onSubmit`.
- El mensaje de éxito se consigue con **otro estado** (`enviado`) y un `return` anticipado, igual que hacía la alerta con `visible`.

8.4 Validación sencilla

Los errores de validación son **datos derivados** (sección 6): se calculan a partir del estado y no necesitan su propio `useState`.

```

const errorCorreo =
  correo === "" ? "El correo es obligatorio"
  : !correo.includes("@") ? "Falta el símbolo @"
  : null;

// En el JSX, bajo el campo:
// {errorCorreo && <p role="alert">{errorCorreo}</p>}

```

8.5 Lo que queda para el capítulo 9

Cuando el formulario crece, repetir un estado y un manejador por cada campo resulta tedioso. En el un tema futuro se estudiarán las soluciones: leer los datos con `FormData`, las acciones de servidor, los *hooks* `useFormStatus` y `useActionState` y la biblioteca `React Hook Form`. Todas se apoyan en lo que has visto aquí.

9. Elevar el estado (*lifting state up*)

Cuando dos componentes necesitan compartir un dato, el estado se coloca en su **ancestro común más cercano** y se reparte hacia abajo mediante *props*: el valor baja como dato y las funciones para modificarlo bajan como eventos propios (los que vimos en UT 02.2 con `onClose`).

9.1 El problema

Resumen debe mostrar el término que se escribe en BarraBusqueda, pero el estado está dentro de BarraBusqueda y Resumen no puede leerlo:

```
function BarraBusqueda() {
  const [termino, setTermino] = useState("");
  return <input type="search" onChange={(e) => setTermino(e.target.value)} />;
}

function Resumen() {
  return <p>Buscando {termino}</p>; // ❌ Error: no se encuentra el nombre 'termino'
}
```

En React los datos fluyen **siempre hacia abajo**, de padre a hijo: un componente no puede leer el estado de un hermano.

9.2 La solución: subir el estado al padre

```
interface PropsBarraBusqueda {
  termino: string;
  alCambiarTermino: (nuevoTermino: string) => void; // evento propio tipado
}

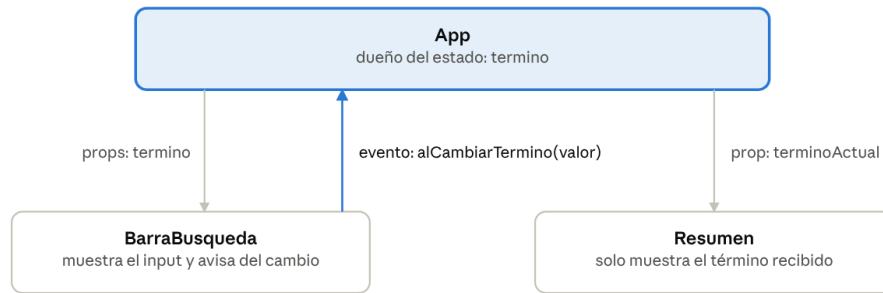
function BarraBusqueda({ termino, alCambiarTermino }: PropsBarraBusqueda) {
  return (
    <input
      type="search"
      value={termino}
      onChange={(e) => alCambiarTermino(e.target.value)}
    />
  );
}

function Resumen({ terminoActual }: { terminoActual: string }) {
  return <p>Buscando: {terminoActual || "(nada todavía)"}</p>;
}

function App() {
  const [termino, setTermino] = useState(""); // el estado vive en el ancestro común

  return (
    <>
      <BarraBusqueda termino={termino} alCambiarTermino={setTermino} />
      <Resumen terminoActual={termino} />
    </>
  );
}
```

El estado vive en el ancestro común: los datos bajan y los eventos suben



estado elevado · App, BarraBusqueda y Resumen

Las flechas grises son datos (*props*) que bajan; la flecha de color es el evento que sube cuando el usuario escribe. Cuando cambia el estado de App, React vuelve a ejecutar App y sus hijos, que reciben el valor nuevo.

9.3 Pasos para elevar el estado

1. Identifica **qué componentes** necesitan leer o modificar el mismo dato.
2. Localiza su **ancestro común más cercano**.
3. **Mueve** el useState a ese ancestro.
4. Pasa el **valor** a los hijos por *props* y una **función** (un evento propio) a los hijos que deben modificarlo.
5. Convierte los hijos en componentes que **muestran** lo que reciben, sin estado propio para ese dato.

9.4 ¿Y si hay muchos niveles?

Elevar el estado funciona muy bien con uno o dos niveles de distancia. Cuando hay que pasar el mismo dato por muchos componentes intermedios que no lo usan (*prop drilling*), existen otras soluciones: **Context** y bibliotecas como **Zustand**. Se estudian en próximos temas, donde verás también cuándo conviene usar cada una.

10. Errores frecuentes, actividades y resumen

10.1 Errores frecuentes

Error	Síntoma	Corrección
<code>onClick={manejar() }</code>	La función se ejecuta al renderizar	<code>onClick={manejar}</code> o <code>onClick={() => manejar(x)}</code>
Usar una variable normal (<code>let</code>) en vez de estado	La pantalla no cambia	<code>useState</code>
Modificar el estado (<code>push</code> , <code>obj.x = 1</code>)	La interfaz no se actualiza	Crear una copia: <code>[...lista, x], { ...obj, x: 1 }</code>
Leer el estado justo después de <code>setX</code>	Se ve el valor antiguo	Recordar que el estado es una instantánea; usar una variable local con el valor nuevo

Error	Síntoma	Corrección
setX(x + 1) varias veces seguidas	Solo suma 1	setX((anterior) => anterior + 1)
Pasar un objeto parcial a setX	Se pierden propiedades (error de TypeScript)	setX({ ...anterior, campo: nuevo })
Guardar en estado algo que se puede calcular	Datos que dejan de coincidir	Calcularlo en el cuerpo del componente
Copiar una <i>prop</i> al estado	No se actualiza cuando el padre la cambia	Usar la <i>prop</i> directamente
Llamar a setX durante el renderizado	Bucle infinito ("Too many re-renders")	Llamarlo solo desde manejadores de eventos
value sin onChange	Campo de solo lectura	Añadir onChange o usar defaultValue
useState([]) sin tipo	TypeScript infiere never[]	useState<Tarea[]>([])
Olvidar e.preventDefault() en un formulario	La página se recarga al enviar	Llamarlo al principio de manejarEnvio

10.2 Buenas prácticas

- Guarda en el estado **lo mínimo posible** y calcula el resto.
 - Trata el estado como **inmutable** y usa la forma funcional cuando dependas del valor anterior.
 - Deja que TypeScript **infiera** y anota solo cuando haga falta.
 - Pon el estado en el **componente más bajo** que lo necesite y súbelo solo cuando haya que compartirlo.
 - Si un componente acumula muchos estados, **divídelo** en componentes más pequeños.
 - Nombra con claridad: valor / setValor, manejarXxx, alXxx o onXxx para las *props* de función.
1. **Formulario de suscripción.** Campos: nombre, correo y casilla de aceptación. Muestra los errores de validación como datos derivados, deshabilita el botón hasta que sea válido y muestra un mensaje de éxito al enviar. (*sección 8*)
 2. **Buscador en dos componentes.** Divide la actividad 4 en BarraBusqueda y ListaProductos, elevando el estado a App. Típa las *props* y el evento propio. (*sección 9*)

10.3 Preguntas de repaso

1. ¿Por qué el siguiente componente no actualiza la pantalla al hacer clic? `let cuenta = 0; <button onClick={() => { cuenta++; }}>{cuenta}</button>`
2. En un clic, el código `setCuenta(cuenta + 1); console.log(cuenta);` se ejecuta con `cuenta` valiendo 3. ¿Qué imprime la consola y cuánto valdrá `cuenta` en el siguiente renderizado?
3. ¿Qué diferencia hay entre `onClick={borrar}` y `onClick={borrar()}?`
4. ¿Por qué `tareas.push("x"); setTareas(tareas);` no actualiza la pantalla?
5. Una tarjeta tiene un `onClick` y dentro un botón con otro `onClick`. ¿Qué ocurre al pulsar el botón y cómo se evita que se ejecute el de la tarjeta?

Respuestas. (1) Porque cuenta es una variable normal: React no se entera de que cambia ni vuelve a ejecutar el componente; hay que usar `useState`. (2) La consola imprime 3 (la instantánea no cambia) y cuenta valdrá 4 en el siguiente renderizado. (3) `borrar` pasa la función para que React la llame cuando ocurra el evento; `borrar()` la ejecuta al renderizar y pasa su resultado. (4) Porque se modifica el mismo array y la referencia no cambia: React compara referencias y no detecta cambio; hay que crear una copia con `setTareas([...tareass, "x"])`. (5) Se ejecuta primero el manejador del botón y después, por burbujeo, el de la tarjeta; se evita con `e.stopPropagation()` en el manejador del botón.

10.4 Resumen y conclusiones clave

- Una variable normal no puede cambiar la pantalla: hace falta **estado**, un dato que React recuerda y que, al cambiar, provoca un nuevo renderizado.
- Renderizar es ejecutar la función del componente. El estado es una **instantánea** por ejecución: `setX` no cambia el valor que ya tienes, sino el del siguiente renderizado.
- Nunca modifiques el estado: crea **copias**. React compara referencias para decidir si debe actualizar La página.
- Si el nuevo valor depende del anterior, usa la **forma funcional** `setX((anterior) => ...)`.
- Guarda como estado solo lo imprescindible; lo que se pueda calcular es un **dato derivado**.
- Los manejadores reciben un objeto evento: `preventDefault()` cancela el comportamiento del navegador y `stopPropagation()` detiene el burbujeo.
- Un campo **controlado** enlaza `value` o `checked` con el estado y lo actualiza con `onChange`.
- Para compartir estado entre componentes, **elevándolo** al ancestro común: los datos bajan por *props* y los eventos suben mediante funciones.

10.5 Próximos pasos

En próximos temas conocerás el resto de *hooks* (`useEffect`, `useReducer`, `useRef`, `useMemo`, `useCallback`), que se apoyan en los conceptos de renderizado, instantáneas e inmutabilidad vistos aquí. También los formularios avanzados y la gestión del estado compartido ampliarán lo estudiado en las secciones 8 y 9.

Anexo. Refuerzo del tipado de eventos

Este anexo amplía lo visto en UT 02.3 sobre eventos tipados. Sirve como hoja de consulta y como conjunto de ejercicios de repaso.

A.1 Regla de oro: inferir o anotar

TypeScript necesita saber qué evento recibe un manejador, y lo averigua según dónde lo escribas:

Situación	¿Hay que anotar el tipo?	Ejemplo
Manejador en línea en el JSX	No, se infiere por el contexto	<code><input onChange={(e) => setTexto(e.target.value)} /></code>
Manejador declarado aparte (function)	Sí, se anota el parámetro	<code>function manejarCambio(e: ChangeEvent<HTMLInputElement>)</code>
Manejador guardado en una constante con tipo Handler	No, el tipo viene del Handler	<code>const manejarClic: MouseEventHandler<HTMLButt</code>

Situación	¿Hay que anotar el tipo?	Ejemplo
		<code>onElement> = (e) => {...}</code>

```
import type { ChangeEvent } from "react";
```

```
// En línea: 'e' ya es ChangeEvent<HTMLInputElement>
<input onChange={(e) => console.log(e.target.value)} />
```

```
// Aparte: sin anotar, 'e' sería 'any' implícito y el compilador se quejaría
function manejarCambio(e: ChangeEvent<HTMLInputElement>) {
  console.log(e.target.value);
}
<input onChange={manejarCambio} />
```

Si usas la plantilla react-ts de Vite, recuerda importar los tipos con la palabra clave type (`import type { ChangeEvent } from "react"` o `import { useState, type ChangeEvent } from "react"`).

A.2 Anatomía de un tipo de evento

Todos los tipos de evento de React siguen el mismo molde:

```
React.MouseEvent<HTMLButtonElement>
```

```
//   ↑ tipo de evento   ↑ elemento al que se une el manejador
```

- La **primera parte** indica la clase de evento (`MouseEvent`, `ChangeEvent`, `KeyboardEvent`, `FocusEvent`, `FormEvent`...).
- El **parámetro genérico** indica el elemento HTML donde está el manejador. De él depende el tipo de `e.currentTarget` (y, en `ChangeEvent` y `FocusEvent`, también el de `e.target`).

Propiedad	Qué tipo tiene	Cuándo usarla
<code>e.currentTarget</code>	El elemento del genérico (<code>HTMLButtonElement</code> , <code>HTMLFormElement</code> ...)	Casi siempre que necesites el elemento que tiene el manejador
<code>e.target</code> en <code>ChangeEvent</code> / <code>FocusEvent</code>	El elemento del genérico (<code>HTMLInputElement</code> ...)	Para leer <code>value</code> o <code>checked</code>
<code>e.target</code> en <code>MouseEvent</code> , <code>KeyboardEvent</code> ...	<code>EventTarget</code> genérico (sin <code>value</code> , sin <code>textContent</code>)	Rara vez; hay que afirmar el tipo: <code>e.target as HTMLElement</code>

A.3 Tabla de referencia

Qué quieres hacer	Propiedad JSX	Tipo del evento	Tipo del manejador
Clic en botón o enlace	<code>onClick</code>	<code>MouseEvent<HTMLButtonElement></code>	<code>MouseEventHandler<HTMLButtonElement></code>
Escribir en un input	<code>onChange</code>	<code>ChangeEvent<HTMLInputElement></code>	<code>ChangeEventHandler<HTMLInputElement></code>
Escribir en un textarea	<code>onChange</code>	<code>ChangeEvent<HTMLTextAreaElement></code>	<code>ChangeEventHandler<HTMLTextAreaElement></code>

Qué quieres hacer	Propiedad JSX	Tipo del evento	Tipo del manejador
			>
Elegir en un select	onChange	ChangeEvent<HTMLSelectElement>	ChangeEventHandler<HTMLSelectElement>
Enviar un formulario	onSubmit	FormEvent<HTMLFormElement>	FormEventHandler<HTMLFormElement>
Pulsar una tecla	onKeyDown	KeyboardEvent<HTMLInputElement>	KeyboardEventHandler<HTMLInputElement>
Perder o ganar el foco	onBlur, onFocus	FocusEvent<HTMLInputElement>	FocusEventHandler<HTMLInputElement>
Pasar el ratón	onMouseEnter	MouseEvent<HTMLDivElement>	MouseEventHandler<HTMLDivElement>

La regla de nombres es sencilla: el tipo del manejador es el del evento con el sufijo Handler.

A.4 Tres formas de tipar un manejador

```
import type { ChangeEvent, ChangeEventHandler } from "react";

function Ejemplo() {
  // 1) Anotando el parámetro (la más habitual)
  function manejarA(e: ChangeEvent<HTMLInputElement>) {
    console.log(e.target.value);
  }

  // 2) Anotando la constante con el tipo Handler: el parámetro se infiere
  const manejarB: ChangeEventHandler<HTMLInputElement> = (e) => {
    console.log(e.target.value);
  };

  // 3) En línea: todo se infiere
  return (
    <>
      <input onChange={manejarA} />
      <input onChange={manejarB} />
      <input onChange={(e) => console.log(e.target.value)} />
    </>
  );
}
```

A.5 Tipar props que reciben manejadores

Cuando un componente propio recibe una función para un evento, hay dos estilos. Para componentes de tu aplicación suele ser mejor el segundo, porque el componente padre no necesita conocer el evento:

```
// Estilo A: se pasa el evento completo (el padre lo ve entero)
interface PropsBotonA {
  alPulsar: React.MouseEventHandler<HTMLButtonElement>;
}

// Estilo B: se pasa solo el dato que interesa (recomendado)
```

```
interface PropsCampoTexto {
  valor: string;
  alCambiar: (nuevoValor: string) => void;
}

function CampoTexto({ valor, alCambiar }: PropsCampoTexto) {
  return <input value={valor} onChange={(e) => alCambiar(e.target.value)} />;
}

// Uso: no hay que tipar ningun evento en el padre
<CampoTexto valor={nombre} alCambiar={setNombre} />
```

Si lo que quieres es que el componente acepte **todos** los atributos de un elemento HTML con sus tipos correctos (onClick, disabled...), usa `ComponentProps<"button">`, que vimos en UT 02.3.

A.6 Un mismo manejador para varios tipos de elemento

Si un manejador se usa en un `input` y en un `select`, el genérico puede ser una **unión**:

```
function manejarCampo(e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
  const { name, value } = e.target; // 'name' y 'value' existen en ambos
  console.log(name, value);
}
```

Solo podrás usar las propiedades que tengan **todos** los elementos de la unión. Por ejemplo, `checked` existe en `input` pero no en `select`, así que habría que comprobarlo antes.

A.7 Errores frecuentes de tipado

Mensaje o síntoma	Causa	Solución
Parameter 'e' implicitly has an 'any' type	Manejador declarado aparte sin anotar	Anotar: <code>e: ChangeEvent<HTMLInputElement></code>
Property 'value' does not exist on type 'EventTarget'	Se lee <code>e.target.value</code> en un evento que no es <code>ChangeEvent</code>	Usar <code>e.currentTarget.value</code> o el evento correcto
Type '(e: MouseEvent) => void' is not assignable to type 'ChangeEventHandler'	Evento equivocado para la propiedad (<code>onChange</code> espera <code>ChangeEvent</code>)	Cambiar el tipo del parámetro por el que corresponde
Se importa <code>MouseEvent</code> y no coincide con React	Se usa el <code>MouseEvent</code> global del DOM en lugar del de React	Importar <code>type MouseEvent</code> de "react" o escribir <code>React.MouseEvent</code>
Property 'checked' does not exist on type 'HTMLSelectElement'	Propiedad que no existe en el elemento indicado	Usar el elemento correcto (<code>HTMLInputElement</code>)
El evento está como any “para que compile”	Se evita el error en vez de resolverlo	Averiguar el tipo (sección A.8)

Un detalle que confunde al principio: existen **dos familias** de tipos con el mismo nombre. `MouseEvent` a secas es el evento nativo del navegador (el que recibe `addEventListener`) y `React.MouseEvent<T>` es el evento de React (el que reciben los manejadores del JSX). En componentes de React usa siempre el de React.

A.8 Cómo averiguar el tipo cuando no lo recuerdas

1. Escribe el manejador **en línea** (`onChange={(e) => {}}`).
2. Pasa el cursor sobre `e` en Visual Studio Code y copia el tipo que muestra.
3. Si prefieres extraer la función, usa ese tipo en la anotación del parámetro.

Otra opción es pasar el cursor sobre la propiedad (`onChange`) y mirar el tipo que espera.

A.9 Ejercicios

Resuélvelos en un proyecto `react-ts` y comprueba que `npm run build` no muestra errores de tipos.

1. **Completar tipos.** Añade la anotación correcta a cada parámetro `e`:
 - o `function a(e) { setNombre(e.target.value); }` en un `<input>`.
 - o `function b(e) { e.preventDefault(); }` en un `<form>`.
 - o `function c(e) { console.log(e.currentTarget.textContent); }` en un `<button>`.
 - o `function d(e) { if (e.key === "Enter") enviar(); }` en un `<input>`.
2. **Encontrar el error.** ¿Por qué este código no compila y cómo se corrige? `function f(e: React.MouseEvent<HTMLButtonElement>) { setNombre(e.target.value); }`
3. **Un manejador, dos campos.** Crea un formulario con un `input` de nombre y un `select` de curso que usen **el mismo** manejador, tipado con una unión, y un estado `{ nombre: string; curso: string }`.
4. **Tipar una prop.** Crea un componente `CampoNumero` con las props `valor: number` y `alCambiar: (n: number) => void` que use `e.target.valueAsNumber` internamente.

Soluciones

```
// 1.
function a(e: React.ChangeEvent<HTMLInputElement>) { setNombre(e.target.value); }
function b(e: React.FormEvent<HTMLFormElement>) { e.preventDefault(); }
function c(e: React.MouseEvent<HTMLButtonElement>)
{ console.log(e.currentTarget.textContent); }
function d(e: React.KeyboardEvent<HTMLInputElement>) { if (e.key === "Enter") enviar(); }

// 2. En MouseEvent, 'target' es un EventTarget genérico y no tiene 'value'.
// Además, un botón no tiene el valor de un campo de texto. Si de verdad se necesita
// el valor del botón: e.currentTarget.value (HTMLButtonElement sí tiene 'value').

// 3.
interface Ficha { nombre: string; curso: string }
const [ficha, setFicha] = useState<Ficha>({ nombre: "", curso: "1" });

function manejarCampo(e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
  const { name, value } = e.target;
  setFicha((anterior) => ({ ...anterior, [name]: value }));
}
// <input name="nombre" value={ficha.nombre} onChange={manejarCampo} />
// <select name="curso" value={ficha.curso} onChange={manejarCampo}>...</select>

// 4.
```

```
interface PropsCampoNumero {  
  valor: number;  
  alCambiar: (n: number) => void;  
}  
  
function CampoNumero({ valor, alCambiar }: PropsCampoNumero) {  
  return (  
    <input  
      type="number"  
      value={valor}  
      onChange={(e) => alCambiar(e.target.valueAsNumber)}  
    />  
  );  
}
```